

AgroSolution

Como o nosso projeto está organizado nas pastas

Sprint 1 · Cadastro · Login · OTP

Pessoal, esse documento explica de um jeito simples onde fica cada coisa no nosso projeto. A ideia é que ninguém fique perdido procurando arquivos — quando a gente segue o mesmo padrão, o projeto cresce sem virar bagunça.

A imagem geral

Pensa no projeto como um **prédio com 2 apartamentos e um arquivo**:

```
AgroSolution/
├── apps/
│   ├── mobile/    ← o app que o produtor instala no celular
│   └── api/        ← o servidor que conversa com o banco
└── docs/          ← documentos do projeto (modelagem, schema)
```

Pasta	Para que serve
apps/mobile/	O aplicativo de celular . É o que o produtor vai baixar e usar. Telas, botões, formulários — tudo aqui.
apps/api/	O servidor (cérebro) . Recebe pedidos do app, valida senha, guarda no banco, envia e-mail. O celular pergunta pra ele "esse e-mail já existe?" e ele responde.
docs/	Documentos do projeto: modelagem do banco, schema SQL, slides da apresentação.

Por que separar em apps/? Porque o app de celular e o servidor são dois programas diferentes. Cada um tem suas próprias bibliotecas e seu próprio jeito de rodar. Manter eles separados evita confusão.

Dentro do app de celular — apps/mobile/src/

A pasta src (de *source*, "fonte") é onde mora **todo o código do app**. Dentro dela tem várias pastas, cada uma com um propósito bem específico:



screens/

As telas que o usuário vê.

Cada arquivo aqui é uma tela inteira do app. Quando você abre a tela de cadastro no celular, é um arquivo desses sendo exibido.

Exemplos: CadastroScreen.tsx, LoginScreen.tsx, OtpScreen.tsx



components/

Pedacinhos de tela que aparecem em várias telas.

Por exemplo: o botão verde do AgroSolution aparece em todas as telas. Em vez de copiar e colar, a gente cria **um** botão aqui e usa em todo lugar. Se um dia mudar a cor, muda em um arquivo só.

Exemplos: Button.tsx, Input.tsx, Checkbox.tsx



navigation/

O mapa do app.

Diz quais telas existem e como o usuário pula de uma pra outra. Por exemplo: depois do cadastro, vai automaticamente pra tela do código de verificação.

Exemplo: AppNavigator.tsx



services/

Quem fala com o servidor.

Tudo que precisa pedir ou mandar dado para a API passa por aqui. É como o "carteiro" do app — ele leva o cadastro do usuário e traz a resposta.

Exemplos: api.ts (configuração do carteiro), auth.service.ts (função cadastrar, login)

✓ schemas/

Regras de validação dos formulários.

Aqui ficam regras como "o nome precisa ter pelo menos 2 letras", "o e-mail precisa ter @", "a senha precisa ter número e letra". Ficam em um lugar separado para a gente reaproveitar e testar.

Exemplo: cadastro.schema.ts

🔧 utils/

Ferramentas pequenas que fazem uma coisa só.

Funções soltas que ajudam em coisas pontuais. Não desenharam nada na tela.

Exemplos:

- **masks.ts** — coloca a pontuação no CPF enquanto a pessoa digita (000.000.000-00)
- **validators.ts** — confere se um CPF é de verdade ou se a pessoa só digitou números aleatórios

🎨 theme/

A identidade visual do app.

Cores, tamanhos de letra, espaçamentos. Em vez de espalhar o verde do AgroSolution (#0F5132) pelo código, a gente coloca em um lugar só. Se um dia mudar a marca, muda aqui — e o app inteiro acompanha.

Exemplos: colors.ts, typography.ts

🪝 hooks/

Lógica que se repete em várias telas.

Por exemplo: "esse usuário está logado?". Em vez de checar isso dentro de cada tela, a gente cria um *hook* que toda tela pode usar.

Exemplo: useAuth.ts

🖼️ assets/

Imagens, ícones e logos.

Ícone do app, splash screen (a tela que aparece enquanto o app carrega), ilustrações usadas dentro das telas.

Dentro do servidor — apps/api/

Quem ficar com a API vai mexer aqui. A pasta tem:

Pasta	Para que serve
src/	O código do servidor. Onde ficam as rotas (/cadastro, /login), os controles de senha, os envios de e-mail.
migrations/	Arquivos SQL que criam ou alteram tabelas no banco. Toda mudança no banco vira um arquivo aqui — assim qualquer pessoa do time consegue subir o banco do zero, igualzinho.

O banco já tem essas tabelas:

Tabela	O que guarda
usuario	Quem usa o app — nome, e-mail, senha (criptografada), CPF/CNPJ.
dispositivo	O celular do usuário — pra mandar notificações e controlar logins.
sessao	Cada vez que alguém faz login, gera uma sessão aqui (pra poder deslogar depois).
fazenda	Cada fazenda cadastrada — nome, área em hectares, cultura, município, UF.
usuario_fazenda	Liga usuário com fazenda. Diz quem é dono, quem é gerente, quem é contador.

Regra de bolso: onde colocar uma coisa nova?

Se você está com um arquivo na mão e não sabe onde colocar, faz essas perguntas:

Pergunta	Vai pra...
É uma tela inteira ?	screens/
Aparece em mais de uma tela ?	components/
Faz uma conta ou formata texto (sem desenhar nada)?	utils/
Conversa com a API / servidor ?	services/
É uma regra de validação de formulário?	schemas/
É uma cor, fonte ou espaçamento ?	theme/
É uma lógica reutilizável (ex: "está logado")?	hooks/
É uma imagem ou ícone ?	assets/

Combinados do time

- 1. Sempre criar arquivo na pasta certa.** Se ficar em dúvida, pergunta no grupo antes de criar.
- 2. Não copiar e colar código.** Se você vai usar o mesmo trecho em duas telas, ele vira um *component*, uma *util* ou um *hook*.
- 3. Cores e tamanhos sempre vêm do theme/.** Nunca digitar #0F5132 direto na tela — usa o nome (`colors.primary`).
- 4. Validação de formulário sempre com Zod, em schemas/.** Não validar na mão dentro da tela.
- 5. Importação curta com @/.** Use `import { Input } from '@components/Input'` em vez de `'../..../components/Input'`.

Quem está com o quê

Tela / parte	Responsável
Tela de Cadastro	Alana
Tela de Login	a definir
Tela de OTP (código por e-mail)	a definir
API (servidor)	a definir
Schema do banco	já feito 